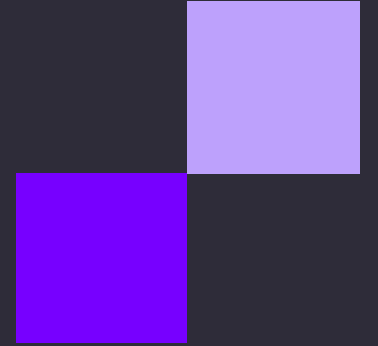




онлайн
университет



ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ



SQL для начинающих

Занятие 13



Представления

Представления (views) в SQL — это виртуальные таблицы, результаты запросов к одной или нескольким таблицам базы данных. Представления не хранят данные физически: вместо этого они сохраняют SQL запрос, который выполняется при обращении к представлению. Это позволяет абстрагироваться от сложных запросов, повторно использовать SQL код и упростить работу с данными для пользователя.

Представления

Представления (views) в SQL — это виртуальные таблицы, результаты запросов к одной или нескольким таблицам базы данных. Представления не хранят данные физически: вместо этого они сохраняют SQL запрос, который выполняется при обращении к представлению. Это позволяет абстрагироваться от сложных запросов, повторно использовать SQL код и упростить работу с данными для пользователя.

Основные особенности представлений:

- **Безопасность данных.** Представления могут ограничивать доступ к определенным столбцам или строкам таблицы, тем самым обеспечивая дополнительный уровень безопасности.
- **Упрощение сложных запросов.** Если вам часто приходится использовать сложные запросы с множеством соединений и подзапросов, представления позволяют скрыть эту сложность, предоставляя простой интерфейс для доступа к результатам таких запросов.
- **Логическая структуризация.** Представления могут использоваться для представления данных в более удобной или логической форме, что упрощает анализ и отчетность.

Представления

Представления (views) в SQL — это виртуальные таблицы, результаты запросов к одной или нескольким таблицам базы данных. Представления не хранят данные физически: вместо этого они сохраняют SQL запрос, который выполняется при обращении к представлению. Это позволяет абстрагироваться от сложных запросов, повторно использовать SQL код и упростить работу с данными для пользователя.

```
CREATE [OR REPLACE] [TEMP | TEMPORARY] VIEW view_name AS  
SELECT columns  
FROM tables  
WHERE conditions;
```

OR REPLACE: Позволяет заменить существующее представление.

TEMP | TEMPORARY: Создает временное представление, доступное только в текущей сессии.

Создадим таблицу и заполним ее

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    status VARCHAR(20) NOT NULL DEFAULT 'active',  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
INSERT INTO users (name, email, status) VALUES  
( 'Иван Иванов', 'ivan.ivanov@example.com', 'active'),  
( 'Пётр Петров', 'petr.petrov@example.com', 'inactive'),  
( 'Сергей Сергеев', 'sergey.sergeev@example.com', 'active'),  
( 'Анна Смирнова', 'anna.smirnova@example.com', 'active'),  
( 'Мария Кузнецова', 'maria.kuznetsova@example.com', 'inactive');
```

Задача 7

Создадим простое представление для вывода активных пользователей

```
CREATE VIEW active_users AS
SELECT id, name, email
FROM users
WHERE status = 'active';
```

	id integer 	name character varying (100) 	email character varying (100) 
1	1	Иван Иванов	ivan.ivanov@example.com
2	3	Сергей Сергеев	sergey.sergeev@example.com
3	4	Анна Смирнова	anna.smirnova@example.com

Обновление данных

Представления могут быть обновляемыми, если они удовлетворяют определенным условиям. Например, простое представление без агрегатных функций и группировок.

Ограничения на обновление

- Представление должно ссылаться на одну таблицу.
- Не должно содержать DISTINCT, GROUP BY, HAVING, LIMIT, OFFSET и некоторые другие конструкции.
- Все столбцы представления должны быть напрямую связаны с столбцами базовой таблицы.

Удаление представления

```
DROP VIEW [IF EXISTS] view_name [CASCADE | RESTRICT];
```

IF EXISTS: Предотвращает ошибку, если представление не существует.

CASCADE: Удаляет объекты, зависящие от представления.

RESTRICT: Запретит удаление, если есть зависимые объекты (по умолчанию).

```
DROP VIEW IF EXISTS active_users;
```


Материализованные представления

Материализованные представления сохраняют результаты запроса на диск, что может значительно ускорить доступ к данным, особенно для сложных и ресурсоемких запросов.

```
CREATE MATERIALIZED VIEW mat_view_name AS  
SELECT columns  
FROM tables  
WHERE conditions  
WITH [NO] DATA;
```

WITH DATA: (по умолчанию) немедленно выполняет запрос и сохраняет данные.

WITH NO DATA: Создает пустое материализованное представление.

Материализованные представления

Данные в материализованном представлении не обновляются автоматически при изменении базовых таблиц. Необходимо выполнить команду REFRESH:

```
REFRESH MATERIALIZED VIEW mat_view_name;
```

Удаление материализованного представления

```
DROP MATERIALIZED VIEW [IF EXISTS] mat_view_name [CASCADE | RESTRICT];
```

Права доступа

Устанавливать права доступа к представлениям можно с помощью команды GRANT:

```
GRANT SELECT, INSERT, UPDATE, DELETE ON view_name TO user_name;
```

Задача 8

Создайте представление `new_users_view`, которое содержит информацию о пользователях, зарегистрировавшихся за последние 30 дней. Включите столбцы: `id`, `name`, `email`, `created_at`.

Задача 8

Создайте представление new_users_view, которое содержит информацию о пользователях, зарегистрировавшихся за последние 30 дней. Включите столбцы: id, name, email, created_at.

```
CREATE VIEW new_users_view AS
SELECT id, name, email, created_at
FROM users
WHERE created_at >= NOW() - INTERVAL '30 days';
```

	id integer	name character varying (100)	email character varying (100)	created_at timestamp without time zone
1	1	Иван Иванов	ivan.ivanov@example.com	2024-11-18 23:20:37.913308
2	2	Пётр Петров	petr.petrov@example.com	2024-11-18 23:20:37.913308
3	3	Сергей Сергеев	sergey.sergeev@example.com	2024-11-18 23:20:37.913308
4	4	Анна Смирнова	anna.smirnova@example.com	2024-11-18 23:20:37.913308
5	5	Мария Кузнецова	maria.kuznetsova@example.com	2024-11-18 23:20:37.913308

Задача 9

Создайте представление `masked_email_view`, в котором адреса электронной почты пользователей частично скрыты для обеспечения конфиденциальности. Отобразите только первые три символа и замените остальные звездочками. Включите столбцы: `id`, `name`, `masked_email`.

	id integer 	name character varying (100) 	masked_email text 
1	1	Иван Иванов	iva*****
2	2	Пётр Петров	pet*****
3	3	Сергей Сергеев	ser*****
4	4	Анна Смирнова	ann*****
5	5	Мария Кузнецова	mar*****

Задача 9

Создайте представление `masked_email_view`, в котором адреса электронной почты пользователей частично скрыты для обеспечения конфиденциальности. Отобразите только первые три символа и замените остальные звездочками. Включите столбцы: `id`, `name`, `masked_email`.

```
CREATE VIEW masked_email_view AS
SELECT
    id,
    name,
    CONCAT(SUBSTRING(email, 1, 3), REPEAT('*', LENGTH(email) - 3)) AS masked_email
FROM users;
```

	id integer	name character varying (100)	masked_email text
1	1	Иван Иванов	iva*****
2	2	Пётр Петров	pet*****
3	3	Сергей Сергеев	ser*****
4	4	Анна Смирнова	ann*****
5	5	Мария Кузнецова	mar*****

Задача 10

Посмотреть код представления

```
SELECT definition  
FROM pg_views  
WHERE viewname = 'masked_email_view';
```

```
SELECT pg_get_viewdef('masked_email_view', true);
```


На этом все на сегодня!